that situation you can use `Results.RedirectToRoute()` to handle the URL generation instead.

NOTE Redirects are more common with server-rendered applications such as Razor Pages, but they're perfectly valid for API applications too.

Listing 6.3 shows how you can return a response from an endpoint that automatically redirects a user to a different named endpoint. The `RedirectToRoute()` method takes the name of the endpoint and any required route parameters, and generates a URL in a similar way to `LinkGenerator`. The minimal API framework automatically sends the generated URL as the response, so you never see the URL in your code. Then the user's browser reads the URL from the response and automatically redirects to the new page.

Listing 6.3 Generating a redirect URL using `Results.RedirectToRoute()`

```
app.MapGet("/test", () => "Hello world!")
    .WithName("hello");                          ❶

app.MapGet("/redirect-me",
    () => Results.RedirectToRoute("hello"))      ❷
```

❶ Annotates the route with the name "hello"

❷ Generates a response that sends a redirect to the "hello" endpoint

By default, `RedirectToRoute()` generates a `302 Found` response and includes the generated URL in the Location response header. You can control the status code used by setting the optional parameters `preserveMethod` and `permanent` as follows:

- `permanent=false`, `preserveMethod=false`—`302 Found`